

第七章 语义分析与中间代码生成

深入浅出知识点总结

Contents

1 全局导读：这章在讲什么？	2
2 中间代码的形式（7.1 节）	2
2.1 为什么需要中间代码？（p.3）	2
2.2 逆波兰表示 / 后缀表示（Postfix Notation）（p.4–5）	2
2.3 三地址码（Three-Address Code, TAC）（p.6–15）	2
2.3.1 四元式（Quadruple）（p.11–12）	3
2.3.2 三元式（Triple）（p.13–14）	3
2.3.3 生成三地址码的语法制导定义（p.15）	3
2.4 图表示：抽象语法树与 DAG（p.16–18）	4
3 声明语句的翻译（7.2 节）	4
3.1 总体思路（p.19–21）	4
3.2 类型表达式（Type Expression）（p.22–25）	4
3.3 类型等价（Type Equivalence）（p.26–28）	4
3.4 过程内声明语句的翻译（p.31–33）	4
3.5 嵌套过程中声明语句的翻译（p.34–54）	5
3.6 记录的翻译（p.55–56）	5
4 赋值语句的翻译（7.3 节）	5
4.1 简单赋值语句（p.57–58）	5
4.2 临时变量的重用（p.59–60）	5
4.3 数组元素的寻址（p.61–73）	5
5 类型检查（7.4 节）	6
5.1 类型检查的规则（p.74–77）	6
5.2 类型转换（Type Conversion）（p.78–79）	6
6 控制结构的翻译（7.5 节）——本章重点难点	6
6.1 布尔表达式的两种翻译方式（p.81–82）	6
6.2 用数值表示布尔值的翻译（p.83–89）	6
6.3 布尔表达式的控制流翻译（p.90–96）	7
6.4 常见控制语句的翻译（p.97–109）	7
6.4.1 if-then 语句（p.100）	7
6.4.2 if-then-else 语句（p.101）	7
6.4.3 while-do 语句（p.102）	8
6.4.4 语句序列 $S_1; S_2$ （p.103）	8
6.5 混合模式的布尔表达式翻译（p.110–116）	8

7 回填技术 (Backpatching) (7.6 节) ——核心难点	8
7.1 为什么需要回填? (p.117–119)	8
7.2 三个核心函数 (p.120)	9
7.3 布尔表达式的回填式翻译 (p.121–130)	9
7.4 常见控制结构的回填式翻译 (p.131–146)	9
7.4.1 if-then (p.132)	10
7.4.2 if-then-else (p.133)	10
7.4.3 while-do (p.134)	10
7.4.4 语句序列 (p.135)	10
8 本章小结 (p.163–164)	11

1 全局导读：这章在讲什么？

编译器的工作可以简化为：源代码 → 词法分析 → 语法分析 → 语义分析 + 中间代码生成 → 优化 → 目标代码。

本章处于语法分析之后、代码优化之前，核心任务是：

1. 对程序进行**语义检查**（类型是否匹配、变量是否声明等）；
2. 把源程序翻译成一种**中间代码**（Intermediate Code），它不依赖具体机器，便于后续优化和移植。

小白理解

你可以把中间代码想象成一种“万能翻译稿”——不管你最终要翻译成 x86 汇编还是 ARM 汇编，都先翻译成这份中间稿，再各自转化，这样编译器只需写一次前端。

本章重点 (p.1)：三地址码（Three-Address Code）、各种语句的目标代码结构、语法制导定义（Syntax-Directed Definition, SDD）与翻译模式（Translation Scheme）。

本章难点 (p.1)：布尔表达式（Boolean Expression）的翻译、控制结构的翻译、回填（Backpatching）技术。

2 中间代码的形式（7.1 节）

2.1 为什么需要中间代码？(p.3)

中间代码的优点：

- 形式简单、语义明确、与目标语言无关；
- 便于编译系统的实现、移植和代码优化。

常见的中间代码形式包括：逆波兰表示（Reverse Polish Notation）、三地址码（Three-Address Code）、抽象语法树（Abstract Syntax Tree, AST）、DAG 图（Directed Acyclic Graph）。

2.2 逆波兰表示 / 后缀表示（Postfix Notation）(p.4–5)

核心思想

把运算符放在操作数后面，这样运算顺序就是运算符出现的顺序，不需要括号。

例子 (p.5)：

中缀表示	前缀表示	后缀表示
a+b	+ab	ab+
a*(b+c)	*a+bc	abc+*
(a+b)*(c+d)	*+ab+cd	ab+cd+*
a:=a*b+c*d	:=a+*ab*c*d	aab*c*d*+:=

2.3 三地址码（Three-Address Code, TAC）(p.6–15)

定义

每条指令最多包含三个地址：两个操作数地址 + 一个结果地址。地址可以是：名字、常量、编译器生成的临时变量（Temporary Variable）。

例子 (p.7)：表达式 $a := (-b) * (c+d) - (c+d)$ 的三地址码：

```

t1 := minus b
t2 := c + d
t3 := t1 * t2
t4 := c + d
t5 := t3 - t4
a  := t5

```

三地址码的指令类型 (p.8-9):

1. $x := y \text{ op } z$ ——二元运算赋值
2. $x := \text{op } y$ ——一元运算赋值
3. $x := y$ ——复制指令
4. `goto L` ——无条件跳转
5. `if x goto L / if false x goto L` ——条件跳转
6. `if x relop y goto L` ——关系运算条件跳转
7. `param x, call p,n, return y` ——过程调用与返回
8. $x := y[i], x[i] := y$ ——变址复制 (数组访问)
9. $x := \&y, x := *y, *x := y$ ——地址/指针操作

2.3.1 四元式 (Quadruple) (p.11-12)

四元式由四个域组成: (op, arg1, arg2, result)。

例子 (p.12): $a := (-b) * (c+d) - (c+d)$ 的四元式表示:

	op	arg1	arg2	result
0	minus	b		t1
1	+	c	d	t2
2	*	t1	t2	t3
3	+	c	d	t4
4	-	t3	t4	t5
5	assign	t5		a

2.3.2 三元式 (Triple) (p.13-14)

三元式只有三个域: (op, arg1, arg2), 用三元式的编号来引用中间结果, 从而节省临时变量。

例如 $\text{arg1} = (0)$ 表示 “用第 0 条三元式的结果作为操作数”。

2.3.3 生成三地址码的语法制导定义 (p.15)

核心属性:

- $E.\text{addr}$: 表达式 E 的结果存放地址 (临时变量或变量名)
- $E.\text{code}$: 为 E 生成的代码序列
- $\text{gencode}(\dots)$: 产生一条三地址码指令
- newtemp : 产生一个新的临时变量名

2.4 图表示：抽象语法树与 DAG (p.16–18)

DAG 的优势

DAG（有向无环图）可以自动消除公共子表达式——相同的子表达式只计算一次，共享同一个节点。

例子 (p.17)：表达式 $a+a*(b-c)-(b-c)/d$ 中， $(b-c)$ 出现两次，在 DAG 中只有一个节点。
生成 DAG 的关键：在调用 `mknode` 和 `mkleaf` 时，先检查是否已存在相同节点，若存在则复用。

3 声明语句的翻译 (7.2 节)

3.1 总体思路 (p.19–21)

声明语句的作用：为程序中的变量/常量指定类型。

类型信息的两大用途：

- **类型检查** (Type Checking)：验证运算是否合法；
- **辅助翻译**：确定存储空间大小、数组地址计算、类型转换等。

编译器的任务：在**符号表** (Symbol Table) 中记录每个名字的类型、相对地址等信息。

3.2 类型表达式 (Type Expression) (p.22–25)

类型可以有层次结构，定义如下：

1. 基本类型：boolean, char, integer, real, void
2. 类型名本身也是类型表达式
3. `array(I, T)`：下标集为 I、元素类型为 T 的数组
4. $T_1 \times T_2$ ：笛卡尔积 (Cartesian Product)
5. `record`：记录类型，带命名域
6. `pointer(T)`：指向 T 的指针
7. $D \rightarrow R$ ：函数类型 (定义域 $\rightarrow$ 值域)

例子(p.25):函数 `function f(a,b:char): ^integer` 的类型表达式为 $\text{char} \times \text{char} \rightarrow \text{pointer}(\text{integer})$ 。

3.3 类型等价 (Type Equivalence) (p.26–28)

- **结构等价** (Structural Equivalence)：两个类型的**结构相同**即等价 (即使名字不同)。
- **名字等价** (Name Equivalence)：类型名**完全相同**才等价。

3.4 过程内声明语句的翻译 (p.31–33)

核心机制

维护一个全局变量 `offset` (偏移量)，每声明一个变量，就将其类型和当前 `offset` 写入符号表，然后 `offset += T.width`。

例子 (p.33)：声明 `x:real; i:integer`

- `enter(x, real, 0)`, `offset` 变为 8 (real 占 8 字节)
- `enter(i, integer, 8)`, `offset` 变为 12 (integer 占 4 字节)

3.5 嵌套过程中声明语句的翻译 (p.34–54)

核心思想

每个过程有自己的符号表。遇到嵌套过程时，**暂停外围过程**，创建新符号表。用两个栈来管理：

- `tblptr` 栈：维护当前活跃的符号表指针
- `offset` 栈：维护当前过程的偏移量

关键函数 (p.35)：

- `mktable(previous)`：创建新符号表，`previous` 指向外围表
- `enter(table, name, type, offset)`：在指定表中创建表项
- `addwidth(table, width)`：记录表的总宽度
- `enterproc(table, name, newtable)`：为过程名创建表项，关联其符号表

课件用一个完整的 Pascal 嵌套程序 `sort` (p.37–54) 展示了符号表的逐步构建过程，每遇到一个新的 `proc`，就 `push` 新表；处理完后 `pop` 回外围表。

3.6 记录的翻译 (p.55–56)

为每个 `record` 类型单独构造一张符号表，处理完所有域后，`T.type := record(表指针)`，`T.width := 所有域宽度之和`。

4 赋值语句的翻译 (7.3 节)

4.1 简单赋值语句 (p.57–58)

辅助子程序：`gencode` (生成中间代码)、`newtemp` (新临时变量)、`lookup` (查符号表)。

翻译模式的核心：遇到运算就生成一条三地址码，用临时变量保存中间结果。

4.2 临时变量的重用 (p.59–60)

优化技巧

临时变量的生存期像**嵌套括号**——一个临时变量在被使用后就“死了”，可以被回收。引入计数器 `c`，产生新临时变量时用 `$c` 并 `c++`；临时变量被用作操作数时 `c--`。

4.3 数组元素的寻址 (p.61–73)

一维数组地址公式

$$\text{addr}(A[i]) = \underbrace{(\text{base} - \text{low}_1 \times w)}_{c, \text{ 常量}} + i \times w$$

其中 w 是每个元素的字节数， low_1 是下标下界。

二维数组地址公式 (行优先)

$$\text{addr}(A[i_1, i_2]) = \underbrace{(\text{base} - (\text{low}_1 \times n_2 + \text{low}_2) \times w)}_{c, \text{ 常量}} + (i_1 \times n_2 + i_2) \times w$$

其中 $n_2 = \text{up}_2 - \text{low}_2 + 1$ 是第 2 维的长度。

翻译要点 (p.63–67):

- 改写文法, 让 `id[E]` 作为一个产生式, 使得数组名 `id` 在第一时间可用;
- 属性 `Elist.array` 记录数组名、`Elist.ndim` 记录维数、`Elist.addr` 记录下标计算中间值;
- `limit(array, j)` 返回第 j 维的长度 n_j 。

完整例子 (p.69–73): 设 A 是 10×20 的整型数组, 翻译 `x := A[y, z]`:

```
t1 := y * 20          // i1 * n2
t1 := t1 + z          // i1*n2 + i2
t2 := baseA - 84      // 常量部分 c = base-(1*20+1)*4
t3 := 4 * t1          // 变量部分 * w
t4 := t2[t3]          // 取数组元素
x  := t4
```

5 类型检查 (7.4 节)

5.1 类型检查的规则 (p.74–77)

两种形式:

- **类型综合** (Type Synthesis): 从子表达式的类型自底向上确定父表达式的类型。例: 若 f 的类型为 $s \rightarrow t$ 且 x 的类型为 s , 则 $f(x)$ 的类型为 t 。
- **类型推断** (Type Inference): 根据使用方式来确定类型 (常用于从函数体推断函数类型)。

5.2 类型转换 (Type Conversion) (p.78–79)

当 `integer` 与 `real` 混合运算时, 需要插入显式转换指令 `inttoreal`。

例子 (p.78): `x := y + i * j` (x, y 是 `real`, i, j 是 `integer`)

```
t1 := i int* j          // 整数乘法
t2 := inttoreal t1      // 转换为实数
t3 := y real+ t2        // 实数加法
x  := t3
```

6 控制结构的翻译 (7.5 节) ——本章重点难点

6.1 布尔表达式的两种翻译方式 (p.81–82)

布尔表达式有两个用途:

1. **计算逻辑值**: 如 `x := a < b`, 结果存为 0 或 1;
2. **改变控制流**: 如 `if a < b then ...`, 用“跳到哪里”表示真假。

6.2 用数值表示布尔值的翻译 (p.83–89)

核心思路

对于关系运算 $a < b$, 生成 4 条指令来把结果变成 0 或 1:

```
100: if a<b goto 103
101: t := 0
102: goto 104
103: t := 1
104: ...
```

`and`, `or`, `not` 则像普通算术运算一样处理。

完整例子 (p.89): $a < b$ or $c < d$ and $e < f$ 生成 13 条指令, 最终得到 $t5 := t1$ or $t4$ 。

6.3 布尔表达式的控制流翻译 (p.90–96)

核心思路——真假出口

给布尔表达式 B 两个标号属性:

- B.true: B 为真时跳转到的位置
- B.false: B 为假时跳转到的位置

对于 $a < b$, 只需生成 2 条指令:

```
if a < b goto B.true
goto B.false
```

or 的翻译逻辑 (p.90–91): $B \rightarrow B1$ or $B2$

- $B1$ 为真 $\Rightarrow$ B 立即为真, 所以 $B1.true = B.true$
- $B1$ 为假 $\Rightarrow$ 还需计算 $B2$, 所以 $B1.false = B2$ 的开始
- $B2$ 的真假出口与 B 相同

and 的翻译逻辑 (p.91): $B \rightarrow B1$ and $B2$

- $B1$ 为真 $\Rightarrow$ 还需计算 $B2$, 所以 $B1.true = B2$ 的开始
- $B1$ 为假 $\Rightarrow$ B 立即为假, 所以 $B1.false = B.false$

not 的翻译: 交换真假出口。

例 7.9 (p.93–96): $a < b$ or $c < d$ and $e < f$ 翻译为:

```
    if a < b goto Ltrue      // B1为真, 整体为真
    goto L1
L1: if c < d goto L2         // B1假, 继续看B2(and)
    goto Lfalse             // c < d假, 整体假
L2: if e < f goto Ltrue      // e < f真, 整体真
    goto Lfalse
```

6.4 常见控制语句的翻译 (p.97–109)

目标代码结构——必须牢记的模板

6.4.1 if-then 语句 (p.100)

```
    B.code                // B为真跳B.true, 为假跳S.next
B.true:
    S1.code                // 执行S1
S.next:                   // 继续
```

6.4.2 if-then-else 语句 (p.101)

```
    B.code                // B为真跳B.true, 为假跳B.false
B.true:
    S1.code
    goto S.next           // 跳过else分支
```



```

B.false:
    S2.code
S.next:

```

6.4.3 while-do 语句 (p.102)

```

S.begin:
    B.code          // B为真跳B.true, 为假跳S.next(退出)
B.true:
    S1.code
    goto S.begin    // 回到循环头
S.next:

```

6.4.4 语句序列 $S_1; S_2$ (p.103)

```

    S1.code
S1.next:
    S2.code

```

完整例子 (例 7.10, p.104–109): while $a < b$ do if $c < d$ then $x := y + z$ else $x := y - z$, 最终生成:

```

L1: if a < b goto L2
    goto Lnext
L2: if c < d goto L3
    goto L4
L3: t1 := y + z
    x := t1
    goto L1
L4: t2 := y - z
    x := t2
    goto L1
Lnext:

```

6.5 混合模式的布尔表达式翻译 (p.110–116)

当布尔运算和算术运算混在一起时 (如 $4 + a > b - c$ and d), 需要引入 $E.type$ 属性区分 arith (算术) 和 bool (布尔), 按情况生成不同的代码。

7 回填技术 (Backpatching) (7.6 节) ——核心难点

7.1 为什么需要回填? (p.117–119)

问题

在一遍扫描 (Single Pass) 中, 生成跳转指令时还不知道目标标号在哪。
例如翻译 if $a < b$ goto ??? 时, ??? 的位置要等到后面的代码生成后才能确定。

解决方案:

1. 先生成跳转指令, 目标标号留空 (goto -);
2. 把这些“待填”指令的地址记录到一个链表中;
3. 一旦确定了目标位置, 回填到所有待填指令中。

7.2 三个核心函数 (p.120)

回填函数

- `makelist(i)`: 创建只含 i 的新链表 (i 是四元式序号)
- `merge(p1, p2)`: 合并两个链表
- `backpatch(p, i)`: 把标号 i 回填到链表 p 中所有跳转指令的目标位置

属性:

- `B.truelist`: B 为真时待回填的跳转指令链表
- `B.falselist`: B 为假时待回填的跳转指令链表
- `S.nextlist`: S 执行完后待回填的跳转指令链表
- `M.quad`: 标记非终结符 M 记录的当前四元式序号 ($M \rightarrow \varepsilon$, $M.quad := nextquad$)

7.3 布尔表达式的回填式翻译 (p.121–130)

$B \rightarrow B1 \text{ or } M \ B2$ (p.121):

- `backpatch(B1.falselist, M.quad)` —— $B1$ 为假时跳到 $B2$ 开头
- `B.truelist := merge(B1.truelist, B2.truelist)` —— 两者任一为真都算真
- `B.falselist := B2.falselist` —— 只有 $B2$ 也为假才算假

$B \rightarrow B1 \text{ and } M \ B2$ (p.122):

- `backpatch(B1.truelist, M.quad)` —— $B1$ 为真时跳到 $B2$ 开头
- `B.truelist := B2.truelist`
- `B.falselist := merge(B1.falselist, B2.falselist)`

$B \rightarrow E1 \text{ relop } E2$ (p.124):

- 生成 `if E1 relop E2 goto - (truelist)` 和 `goto - (falselist)`

例 7.11 (p.126–130): $a < b \text{ or } c < d \text{ and } e < f$ 的回填过程:

```
100: if a<b goto ___      // truelist
101: goto ___             // falselist → 回填为102
102: if c<d goto ___      // → 回填为104
103: goto ___             // falselist
104: if e<f goto ___      // truelist
105: goto ___             // falselist
```

最终回填结果: $B.truelist = \{100, 104\}$ (等外层回填), $B.falselist = \{103, 105\}$ 。

7.4 常见控制结构的回填式翻译 (p.131–146)

新增非终结符:

- $M \rightarrow \varepsilon$: 记录当前位置 $M.quad := nextquad$
- $N \rightarrow \varepsilon$: 生成 `goto -` 并记入 $N.nextlist$ (用于跳过 `else` 分支)

7.4.1 if-then (p.132)

$S \rightarrow \text{if } B \text{ then } M \ S1:$

- `backpatch(B.truelist, M.quad)`
- `S.nextlist := merge(B.falselist, S1.nextlist)`

7.4.2 if-then-else (p.133)

$S \rightarrow \text{if } B \text{ then } M1 \ S1 \ N \ \text{else } M2 \ S2:$

- `backpatch(B.truelist, M1.quad)`
- `backpatch(B.falselist, M2.quad)`
- `S.nextlist := merge(S2.nextlist, merge(N.nextlist, S1.nextlist))`

7.4.3 while-do (p.134)

$S \rightarrow \text{while } M1 \ B \ \text{do } M2 \ S1:$

- `backpatch(S1.nextlist, M1.quad)` ——循环体结束后回到条件判断
- `backpatch(B.truelist, M2.quad)` ——条件为真进入循环体
- `S.nextlist := B.falselist` ——条件为假退出
- 生成 `goto M1.quad`

7.4.4 语句序列 (p.135)

$S \rightarrow S1 ; M \ S2:$

- `backpatch(S1.nextlist, M.quad)`
- `S.nextlist := S2.nextlist`

例 7.12 (p.136–146) ——大型综合例题:

翻译 `while a<b do if c<5 then while x>y do z:=x+1 else x:=y`
最终生成的四元式序列:

100: if a<b goto 102	
101: goto 112	(退出外层while)
102: if c<5 goto 104	
103: goto 110	(进入else)
104: if x>y goto 106	
105: goto 100	(内层while假,回外层)
106: t1 := x + 1	
107: z := t1	
108: goto 104	(回内层while头)
109: goto 100	(then分支结束,回外层)
110: x := y	
111: goto 100	(else结束,回外层)
112:	(外层while出口)

8 本章小结 (p.163–164)

1. 中间代码形式：逆波兰表示、三地址码（四元式/三元式）、AST/DAG；
2. 声明语句翻译：在符号表中填入类型和相对地址；嵌套过程用栈管理多个符号表；
3. 赋值语句翻译：核心是算术表达式翻译和数组元素地址计算；
4. 类型检查：类型综合与推断、隐式类型转换；
5. 控制结构翻译：布尔表达式的数值表示 vs 控制流表示（真假出口）；if/while/序列的代码模板；
6. 回填技术：一遍扫描时先留空跳转目标，确定后统一回填。

考试要点提醒

- 能够手工将表达式翻译成三地址码、四元式、三元式
- 能画出 DAG 图
- 能根据语法制导定义手工翻译声明语句、赋值语句
- 能画出 if-then / if-then-else / while-do 的代码结构图
- 能完成布尔表达式的控制流翻译（真假出口）
- 能完成回填式翻译（包括画注释分析树、写出待回填链表、逐步回填）
- 综合大题：给一个嵌套控制语句，用回填技术生成完整四元式序列